

git Cheat Sheet

Track changes to files, undo mistakes, and share work through a remote repository.

Using it

```
git config --global user.name "Mark" ← once per machine (same again for user.email): who your commits are from
git init | git clone https://github.com/org/app.git ← start a new repo here | copy an existing one into app/
your files --add--> staging --commit--> local repo --push--> remote ← git pull brings commits down
```

Command	Does	Command	Does
git add file / .	Stages one file / everything	git diff	Shows changes not yet staged
git commit -m "msg"	Commits what's staged	git diff --staged	Shows what will be committed
git commit -am "msg"	Commits all tracked edits	git show <sha>	Shows one commit's changes

Reading the output: git status -sb and git log --oneline --graph --decorate

```
## main...origin/main [ahead 1] ← on branch main; 1 commit not pushed yet ([behind 2] = 2 not pulled yet)
M  nginx.conf ← left column = staging area: M = modified and staged
M  app.py ← right column = working folder: modified, not staged (MM = staged, then edited again)
A  deploy.sh ← new file, staged (also D = deleted, R = renamed)
?? notes.txt ← untracked: git isn't tracking it yet (git add it, or .gitignore it)
```

```
*    a4884e7 (HEAD -> main) Merge branch 'fix-tls' ← HEAD -> main = you are here, on branch main
| \ ← the lines show branches splitting and joining
| * 5c533aa (fix-tls) Fix TLS cert path ← a commit on branch fix-tls; 5c533aa = its short ID (sha)
* | 1fee863 (tag: v1.2, origin/main) Bump version ← origin/main = the remote, as of your last fetch
| /
```

Branches and remotes

Command	Does	Command	Does
git switch -c fix-tls	New branch, switches to it	git push -u origin main	First push; sets upstream
git switch main	Switches to a branch	git pull	Fetches and merges
git merge fix-tls	Merges it into this branch	git fetch	Downloads, doesn't merge
git branch -d fix-tls	Deletes a merged branch	git rebase main	Replays commits onto main*

* Rebase rewrites history: fine for your own unpushed branch, not for shared ones.

Undoing things

I want to...	Command	Notes
Unstage a file (keep my edits)	git restore --staged file	Opposite of git add
Throw away edits to a file	git restore file	Edits are gone for good
Fix the last commit's message or files	git add file; git commit --amend	Only if not pushed yet
Undo the last commit, keep the changes	git reset --soft HEAD~1	Changes stay staged
Undo a commit that's already pushed	git revert <sha>	Adds a commit that reverses it
Find a 'lost' commit (after a reset etc.)	git reflog	Then git branch rescue <sha>

Stash, .gitignore and tags

Command	Does	.gitignore line	Does
git stash	Shelves uncommitted changes	*.log	Ignores all .log files
git stash pop	Brings them back	build/	Ignores any build/ folder
git tag -a v1.2	Tags the current commit	!keep.log	Exception: do track keep.log
git push origin v1.2	Pushes the tag (not automatic)	git rm --cached f	Untracks f so .gitignore applies

Remember: never **reset --hard** or force-push shared branches unless you mean it; **git reflog** can usually rescue you.